



Phage
SECURITY

PinLink

Security Review

Conducted by:

Pyro, Security Researcher

26.08.2025

Table of Contents

Disclaimer 3

System overview 3

Executive summary 3

 Overview 3

 Timeline 3

 Scope 4

 Issues Found 4

Findings 5

 Informational 5

 [I-01] delist may fail to delist a listing in some conditions 5

 Description 5

 Recommendation 5



Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

PinLink's new product enables users to purchase BTC mining hashrate for a specified duration. Users can buy listings (ERC1155 tokens) and earn native BTC, which is sent directly to their BTC address. These listings are available for purchase, with prices varying based on the hashrate and duration.

Executive summary

Overview

Project Name	PinLink
Repository	private
Commit hash	3c56f25151bc8efa39b1960e3a25f82bb5852371
Remediation	21e0d7a65c073512ff98ee77392a436a2360d19b
Methods	Manual review

Timeline

Audit kick-off	25.08.2025
End of audit	25.08.2025
Remediations start	26.08.2025
Remediations end	26.08.2025

Scope

src/hashlink/hashlinkFutures.sol

src/hashlink/hashlinkMarketplace.sol

Issues Found

Severity	Count
High	0
Medium	0
Low	0
Informational	1



Findings

Informational

[I-01] delist may fail to delist a listing in some conditions

Description

When calling `delist` for all remaining listings, there is a small chance a listing is actually purchased (maliciously or accidentally) right before `delist` gets executed. This will result in the transaction being reverted due to `_availableSupply(tokenId)` being smaller than `amount`. Which may result in another listing being purchased before the next one is made.

```
function delist(uint256 tokenId, uint256 amount) external onlyOwner {
    require(_availableSupply(tokenId) ≥ amount, "Not enough supply to delist");
    listings[tokenId].listedAmount -= amount;
    emit ListingSupplyDecreased(tokenId, amount);
}
```

This would make removing listings, that are no longer supported (due to one reason or another) difficult.

Recommendation

Consider adding an option for when `amount = uint256.max` to `delist _availableSupply(tokenId)`.

Resolution: Fixed in 21e0d7a